

CineVerse Technical Requirements Document (TRD) v2.0

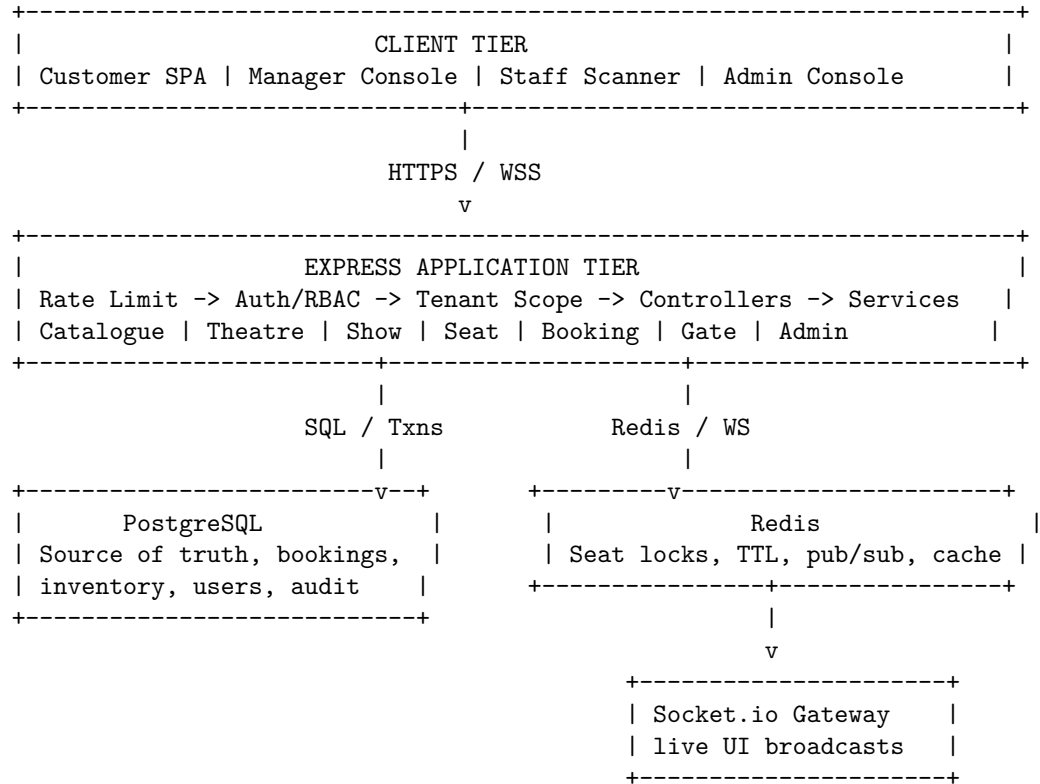
1. Technology baseline

The supplied architecture uses PERN: PostgreSQL, Express.js, React/Vite and Node.js, with Redis for distributed locking/caching and Socket.io for real-time updates. [Source: provided CineVerse specification]

Recommended project stack

Layer	Technology	Purpose
Frontend	React + Vite + TypeScript	Customer, Manager, Staff and Admin applications
Styling	Tailwind CSS	Shared UI system
Client data	TanStack Query	Server-state fetching/cache/invalidation
Local UI state	Zustand or Redux Toolkit	Auth/UI/checkout state
Backend	Node.js + Express + TypeScript	REST API and WebSocket orchestration
Database	PostgreSQL 15+	Source of truth and transactional booking commit
Cache/lock	Redis	5-minute seat locks, sessions/cache, pub/sub
Realtime	Socket.io	Live seat updates and dashboards
Auth	JWT access + HTTP-only refresh token	Claims-based auth and session management
Validation	Zod or Joi	Request/DTO validation
Password hashing	Argon2id or bcrypt cost 12	Credential security
QR	Signed payload + QR library	Ticket generation and scanner verification
Testing	Vitest/Jest + Supertest + Playwright	Unit, API and end-to-end testing

2. Architecture



3. Backend module boundaries

```

src/
  app.ts
  server.ts
  config/
  modules/
    auth/
    users/
    theatres/
    screens/
    seats/
    movies/
    shows/
    pricing/
    bookings/
    payments/
    coupons/

```

```

tickets/
staff/
analytics/
notifications/
audit/
admin/
middleware/
  auth.ts
  permissions.ts
  tenantScope.ts
  rateLimit.ts
  errorHandler.ts
realtime/
  socket.ts
  seatEvents.ts
jobs/
  lockExpiry.ts
  notifications.ts
  reconciliation.ts    # P2 when real payment is enabled
db/
  migrations/
  seed/
shared/
  errors/
  utils/
  types/

```

4. Authentication and authorization

Do not use coarse checks such as `if (role === 'admin')`. The supplied TRD requires explicit permission claims and tenant-scoping middleware. [Source: provided CineVerse specification]

Auth flow

Register/Login

```

-> validate input
-> lookup account
-> verify password / OTP
-> issue 15 min access token
-> store/rotate refresh token
-> attach permission claims
-> request middleware verifies JWT
-> requirePermission(PERMISSION)
-> enforceTenantScope when theatre resource is involved

```

Minimum permissions

CREATE_MOVIE, EDIT_MOVIE, APPROVE_THEATRE, CREATE_SHOW, MANAGE_SEATS, MANAGE_STAFF, VIEW_THEATRE_ANALYTICS, SCAN_TICKET, MANAGE_USERS, MANAGE_COUPONS, VIEW_PLATFORM_ANALYTICS, GLOBAL_OVERRIDE.

5. Real-time seat locking design

The source TRD defines a two-tier strategy: Redis SETNX with a 300-second TTL, then PostgreSQL SELECT ... FOR UPDATE during the final booking commit. [Source: provided CineVerse specification]

Lock key

lock:show:{showId}:seat:{seatId}

Acquisition

SET lock:show:{show_id}:seat:{seat_id} {user_id} NX EX 300

- OK -> lock acquired; publish seat.locked.
- nil -> another user owns lock; return HTTP 409.
- Expiry -> publish seat.available.

Important implementation rule

The Redis lock is a temporary concurrency layer, not the final source of truth. The PostgreSQL transaction must re-check seat state and commit BOOKED state atomically.

6. Booking state machine

```
INITIATED -> PROCESSING -> CONFIRMED
                        |           |
                        +--> FAILED  +--> CANCELLED
                        |
                        +--> EXPIRED
```

For the current P0 release, the external payment gateway is replaced by a Mock Payment Adapter. The state machine remains identical so a real gateway can be plugged in later without redesigning the booking model.

7. Booking transaction

BEGIN;

SELECT *
FROM show_seat_status

```

WHERE show_id = $1
    AND seat_id IN (...)
FOR UPDATE;

-- verify every requested seat is not BOOKED/UNAVAILABLE
-- create booking + booking_seats + payment record
-- mark inventory BOOKED

COMMIT;

```

After commit, delete Redis lock keys and emit `seat.booked` events. The source design uses this exact two-tier approach to prevent duplicate sales. [Source: provided CineVerse specification]

8. API surface

Auth

- POST `/api/v1/auth/register`
- POST `/api/v1/auth/login`
- POST `/api/v1/auth/refresh`
- POST `/api/v1/auth/logout`
- POST `/api/v1/auth/otp/send`
- POST `/api/v1/auth/otp/verify`

Discovery

- GET `/api/v1/movies`
- GET `/api/v1/movies/:id`
- GET `/api/v1/theatres?movieId=&date=&city=`
- GET `/api/v1/shows/:showId`

Seats

- GET `/api/v1/shows/:showId/seats`
- POST `/api/v1/shows/:showId/seats/lock`
- POST `/api/v1/shows/:showId/seats/release`

Bookings

- POST `/api/v1/bookings/initiate`
- POST `/api/v1/bookings/mock-pay`
- POST `/api/v1/bookings/:id/confirm` (internal or mock-payment call-back path)
- GET `/api/v1/bookings/my`
- GET `/api/v1/bookings/:id`
- POST `/api/v1/bookings/:id/cancel`

Staff

- GET /api/v1/staff/today
- POST /api/v1/staff/validate-ticket
- GET /api/v1/staff/booking-lookup

Manager

- POST /api/v1/manager/screens
- POST /api/v1/manager/screens/:id/seats/bulk
- POST /api/v1/manager/shows
- GET /api/v1/manager/bookings
- GET /api/v1/manager/analytics
- POST /api/v1/manager/staff

Admin

- POST /api/v1/admin/movies
- PATCH /api/v1/admin/movies/:id
- GET /api/v1/admin/theatres/pending
- POST /api/v1/admin/theatres/:id/approve
- POST /api/v1/admin/users/:id/block
- GET /api/v1/admin/audit-logs

9. WebSocket events

Event	Producer	Consumers
seat.locked	Seat service	All users viewing the show
seat.released	Lock expiry/service	All users viewing the show
seat.booked	Booking service	Show viewers, manager dashboard
booking.confirmed	Booking service	Booking owner
ticket.used	Staff scanner	Booking owner, manager dashboard
occupancy.updated	Analytics service	Manager/Admin dashboard

10. Error contract

```
{
  "success": false,
  "error": {
    "code": "SEAT_UNAVAILABLE",
    "message": "One or more selected seats are unavailable.",
    "details": ["C7"]
  },
  "requestId": "req_..."
}
```

Use stable machine-readable error codes. Do not expose SQL errors or sensitive internal state to clients.

11. Security controls

- Argon2id or bcrypt cost 12 for passwords.
- HTTP-only refresh cookies where applicable.
- Rate-limit login/OTP endpoints.
- Lock account after repeated failed attempts, with an admin unlock path.
- Validate and sanitize all DTOs.
- Parameterized SQL / ORM bindings only.
- Verify QR signature/hash before ticket validation.
- Audit every privileged mutation.
- Tenant isolation on all manager/staff theatre operations.
- No raw payment credentials stored in CineVerse.

The supplied requirements explicitly require password hashing, HMAC verification for payment callbacks, claims-based RBAC and tenant scope. [Source: provided CineVerse specification]

12. Testing strategy

P0 mandatory tests

1. Auth happy/failure paths.
2. Role permission matrix.
3. Cross-theatre access rejection.
4. Show overlap rejection.
5. Seat lock success and HTTP 409 conflict.
6. Lock expiry releases seat.
7. Two users compete for the same seat.
8. Transaction commits only available seats.
9. Mock payment idempotency.
10. QR ticket generation.
11. Six-point gate validation.
12. Second scan rejection.
13. Cancellation cutoff calculations.
14. Audit log creation.

Concurrency test

Run at least 50-100 concurrent attempts against the same seat for development validation and prove exactly one booking wins. Later, stress test toward the source target of 5,000+ concurrent checkout attempts. The 5,000+ goal belongs to production hardening rather than P0 feature completion.

13. Deployment baseline

Frontend -> Vercel / static hosting
Backend -> Node process / VPS / container
Postgres -> managed PostgreSQL or VPS PostgreSQL
Redis -> managed Redis / local container
Storage -> object storage for posters/profile images

Environment variables should include database URL, Redis URL, JWT secret, refresh secret, object-storage credentials, QR secret, and future payment gateway keys. Payment gateway keys must remain optional until P2.

14. Operational readiness

Before demo/release:

- Database migrations reproducible from empty database.
- Seed script creates roles, sample admin, one theatre, one screen, seats, movies, shows and test coupons.
- Health endpoints exist: `/health`, `/health/db`, `/health/redis`.
- Structured logs contain requestId.
- Error tracking exists at the application boundary.
- Backups configured for PostgreSQL.

Document status: Project working baseline. This package preserves the supplied CineVerse requirements and explicitly separates current MVP priorities from deferred capabilities.

Source basis: CineVerse Role & Functionality Specification v1.0 (provided PDF); Pasted text containing the PRD/TRD/flow/schema/implementation baseline (provided file). No external research was used.